

TP — Créer une API Express de zéro : « Bibliothèque »

Séance du jeudi 1er octobre 2026 · Stack : Express 5, TypeScript, Prisma 7, PostgreSQL, Zod 4, bcrypt, JWT, Multer

Support de cours associé : **Cours — Monter une API Express + TypeScript + Prisma pas à pas** (docs/COURS-api-express-prisma.md). Le cours construit une API de *notes* ; ce TP vous demande d'appliquer **exactement la même démarche** à un autre sujet. Ne copiez pas le cours : adaptez-le.

1. Contexte et objectifs

Jusqu'ici vous avez travaillé sur `mkp_localhost`, un projet déjà configuré. Aujourd'hui vous partez d'un **dossier vide** et d'une **base de données vide**. À la fin du TP, vous devez être capables de :

- créer une base PostgreSQL et un utilisateur dédiés à un projet ;
- initialiser un projet Node + TypeScript et comprendre chaque fichier de configuration ;
- brancher Prisma 7 (schéma, migrations, client généré, adapter `pg`) ;
- structurer une API Express par **modules** (routes / contrôleur / schémas) ;
- centraliser les erreurs et valider toutes les entrées avec Zod ;
- sécuriser des routes avec des JWT (access + refresh) ;
- gérer l'upload d'une image avec Multer.

2. Prérequis

Outil	Version minimale	Vérification
Node.js	22 LTS (24 conseillé)	<code>node -v</code>
npm	10	<code>npm -v</code>
PostgreSQL	15	<code>psql --version</code>
Git	—	<code>git --version</code>
VS Code + extension REST Client (ou Postman)	—	—

Dépendances imposées (mêmes technologies que `mkp_localhost`) :

- **dépendances** : `express`, `@prisma/client`, `@prisma/adapters-pg`, `dotenv`, `zod`, `bcrypt`, `jsonwebtoken`, `multer`
- **dépendances de dev** : `typescript`, `tsx`, `prisma`, `@types/node`, `@types/express`, `@types/bcrypt`, `@types/jsonwebtoken`, `@types/multer`

3. Le sujet : API « Bibliothèque »

Une petite bibliothèque veut une API pour gérer ses **auteurs**, ses **livres** et les **emprunts** de ses membres.

3.1 Modèle de données attendu

User 1 — n Loan n — 1 Book n — 1 Author

Modèle	Champs obligatoires	Remarques
User	<code>id</code> (uuid), <code>first_name</code> , <code>last_name</code> , <code>email</code> (unique), <code>password</code> , <code>role</code> (MEMBER ADMIN, défaut MEMBER), <code>profile_picture?</code> , <code>refreshToken?</code> , <code>createdAt</code> , <code>updatedAt</code>	<code>role</code> est une enum Prisma
Author	<code>id</code> , <code>first_name</code> , <code>last_name</code> , <code>birth_year?</code> , <code>createdAt</code> , <code>updatedAt</code>	un auteur a plusieurs livres
Book	<code>id</code> , <code>title</code> , <code>isbn</code> (unique), <code>published_year</code> , <code>stock</code> (Int, défaut 1), <code>cover?</code> , <code>authorId</code> , <code>createdAt</code> , <code>updatedAt</code>	<code>cover</code> = chemin de l'image de couverture
Loan	<code>id</code> , <code>userId</code> , <code>bookId</code> , <code>borrowedAt</code> (défaut <code>now()</code>), <code>dueAt</code> , <code>returnedAt?</code>	<code>returnedAt</code> = null ⇒ emprunt en cours

Règles de suppression :

- supprimer un **auteur** qui a encore des livres doit être **refusé** (409) ;
- supprimer un **utilisateur** supprime ses emprunts (`onDelete: Cascade`).

3.2 Routes à livrer

Toutes les routes sont préfixées par `/api`. Sécurité : = access token obligatoire, [admin] = rôle ADMIN obligatoire.

Santé

Méthode	Route	Comportement
GET	<code>/api/health</code>	200 { <code>success: true</code> , <code>message: "API is running"</code> }

Authentification — /api/auth

Méthode	Route	Body	Comportement
POST	/register	first_name, last_name, email, password	409 si email déjà pris. Mot de passe haché avec bcrypt. 201 avec l'utilisateur sans password.
POST	/login	email, password	401 « Invalid credentials » (même message si email inconnu ou mauvais mot de passe). 200 avec { accessToken, refreshToken, user }.
POST	/refresh-token	refreshToken	Vérifie la signature JWT et que le token correspond à celui stocké (haché) en base. 200 avec un nouveau couple de tokens. L'ancien refresh token ne doit plus fonctionner.
POST	/logout Sécurité :	—	Efface le refresh token en base.
GET	/me Sécurité :	—	200 avec l'utilisateur courant (sans password).

Utilisateurs — /api/users

Méthode	Route	Comportement
PATCH	/me/avatar Sécurité :	Upload multipart/form-data, champ avatar. jpeg/png/webp uniquement, 2 Mo max. L'ancienne image est supprimée du disque.

Auteurs — /api/authors

Méthode	Route	Comportement
GET	/	Liste des auteurs avec le nombre de livres de chacun (_count).
GET	/:id	Détail d'un auteur avec ses livres . 404 si inconnu, 400 si l'id n'est pas un uuid.
POST	/ [auth, admin]	Création.
PATCH	/:id [auth, admin]	Mise à jour partielle (au moins un champ).
DELETE	/:id [auth, admin]	409 si l'auteur a encore des livres.

Livres — /api/books

Méthode	Route	Comportement
GET	/	Liste paginée : query <code>?page=1&limit=10&search=...</code> (recherche sur le titre, insensible à la casse). Réponse : <code>{ data, meta: { page, limit, total } }</code> . Chaque livre inclut son auteur.
GET	<code>/:id</code>	Détail avec l'auteur.
POST	<code>/ [auth, admin]</code>	Création. 404 si <code>authorId</code> n'existe pas, 409 si l'ISBN existe déjà.
PATCH	<code>/:id [auth, admin]</code>	Mise à jour partielle.
DELETE	<code>/:id [auth, admin]</code>	Suppression.
PATCH	<code>/:id/cover [auth, admin]</code>	Upload de la couverture, champ <code>cover</code> (mêmes règles que l'avatar).

Emprunts — `/api/loans`

Méthode	Route	Comportement
POST	/ Sécurité :	Body <code>{ bookId }</code> . 409 si <code>stock = 0</code> ou si l'utilisateur a déjà 3 emprunts en cours . Crée l'emprunt avec <code>dueAt = maintenant + 14 jours</code> et décrémente <code>stock</code> — les deux opérations dans une transaction (<code>prisma.\$transaction</code>).
GET	<code>/me</code> Sécurité :	Emprunts de l'utilisateur connecté (en cours d'abord), avec le livre.
PATCH	<code>/:id/return</code> Sécurité :	Retour du livre : <code>returnedAt = now()</code> et <code>stock + 1</code> (transaction). 404 si l'emprunt n'est pas à l'utilisateur, 409 s'il est déjà rendu.

4. Travail demandé, étape par étape

Suivez les étapes du cours dans l'ordre. Faites **un commit par étape** avec un message clair.

- Base de données** — Créez un utilisateur PostgreSQL `library_user` (avec le droit `CREATEDB`) et une base `library_db` qui lui appartient. Vérifiez la connexion avec `psql`.
- Projet** — `git init`, `npm init -y`, `.gitignore` complet, installation des dépendances listées en §2.
- TypeScript** — `tsconfig.json` (strict), scripts `dev`, `build`, `start`, `typecheck`. `npm run typecheck` doit passer.
- Environnement** — `.env` (non commité) et `.env.example` (commité), plus un module

- `src/config/env.ts` qui plante au démarrage si une variable obligatoire manque.
5. **Prisma** — `prisma.config.ts`, `schema.prisma` avec les 4 modèles et l'enum `Rôle`, première migration `init`, client singleton dans `src/config/prisma.ts`.
 6. **Express** — `app.ts` (configuration) séparé de `server.ts` (démarrage). Route `/api/health`.
 7. **Erreurs** — classe `AppError`, middleware `notFound`, middleware `errorHandler` qui gère : `ZodError` (400), `AppError`, Prisma `P2002` (409) et `P2025` (404), erreurs JWT (401), `MulterError` (400), JSON mal formé (400), autres (500 sans détail en production).
 8. **Validation** — un schéma Zod par route qui reçoit des données (body, params **et** query) et un middleware `validate`.
 9. **Modules CRUD** — `authors` puis `books` (routes / contrôleur / schémas).
 10. **Authentification** — module `auth`, middleware `authenticate`, puis un middleware `authorize("ADMIN")` à écrire vous-mêmes.
 11. **Upload** — configuration Multer, avatar puis couverture de livre, fichiers servis sur `/uploads`.
 12. **Emprunts** — module `loans` avec transactions.
 13. **Tests** — fichier `requests.http` qui rejoue le scénario du §6.

5. Contraintes techniques (vérifiées à la correction)

- **Typage strict** : `"strict": true`, **aucun any**, `npm run typecheck` sans erreur.
- **Arborescence par modules** : `src/modules/<nom>/<nom>.routes.ts`, `<nom>.controller.ts`, `schemas/`.
- **Aucun `res.status(4xx)` dans les contrôleurs** : on lève une `AppError` (ou on laisse remonter l'erreur) et c'est `errorHandler` qui répond.
- **Format de réponse unique** :
 - succès : `{ success: true, message, data?, meta? }`
 - erreur : `{ success: false, message, errors? }`
- **Le champ `password` ne sort jamais** d'une réponse (utilisez `select` ou `omit`).
- Le refresh token est stocké **haché** en base.
- **Aucun secret dans Git** : `.env` ignoré, `.env.example` à jour.
- `src/generated/` et `uploads/` sont dans le `.gitignore` ; un `postinstall` régénère le client Prisma.
- Un seul `PrismaClient` dans toute l'application.
- Le premier compte ADMIN est créé par un **script de seed** (`prisma/seed.ts`, lancé avec `npx tsx prisma/seed.ts`) — pas par une route publique.

6. Scénario de test attendu (`requests.http`)

1. GET `/api/health` → 200
2. POST `/api/auth/register` (body vide) → 400 avec le tableau `errors`
3. POST `/api/auth/register` → 201, sans password dans la réponse
4. POST `/api/auth/register` (même email) → 409
5. POST `/api/auth/login` (mauvais mot de passe) → 401
6. POST `/api/auth/login` (membre) → 200 + tokens
7. GET `/api/auth/me` sans token → 401 ; avec token → 200
8. POST `/api/authors` avec le token **membre** → 403
9. POST `/api/auth/login` (admin créé par le seed) → 200

10. `POST /api/authors` (admin) → 201
11. `POST /api/books` (admin) avec un `authorId` inconnu → 404 ; valide → 201 ; même ISBN → 409
12. `PATCH /api/books/:id/cover` avec un `.txt` → 400 ; avec un `.png` → 200, image accessible sur `/uploads/...`
13. `GET /api/books?page=1&limit=5&search=...` → 200 avec meta
14. `POST /api/loans` (membre) → 201 et le stock du livre a diminué de 1
15. `PATCH /api/loans/:id/return` → 200, stock remis ; une 2^e fois → 409
16. `DELETE /api/authors/:id` alors qu'il a des livres → 409
17. `POST /api/auth/refresh-token` → 200 ; le même refresh token une 2^e fois → 401
18. `GET /api/nimportequoi` → 404 au format standard

7. Livrables et modalités de rendu

- Un **dépôt GitHub** `library-api-<prenom-nom>` contenant :
 - le code source, **un commit par étape** (au moins 10 commits) ;
 - le dossier `prisma/migrations/` ;
 - le fichier `requests.http` ;
 - un `README.md` qui explique : comment créer la base, installer, lancer le projet, lancer le seed, et vos choix techniques (2-3 lignes chacun).
- Le projet doit démarrer avec uniquement : `npm install` → copier `.env.example` en `.env` → `npx prisma migrate deploy` → `npx tsx prisma/seed.ts` → `npm run dev`.

Date limite : à préciser par le formateur.

Barème indicatif (/20)

Critère	Points
Base de données, configuration projet, TypeScript, <code>.env</code>	3
Schéma Prisma (relations, enum, contraintes) et migrations	3
Gestion centralisée des erreurs + validation Zod	3
CRUD auteurs / livres (pagination, recherche, include)	3
Authentification JWT (access/refresh, rotation, rôles)	4
Upload (avatar + couverture)	2
Emprunts avec transactions	1
Qualité : typecheck, commits, README, <code>requests.http</code>	1

Bonus

- Middleware de pagination réutilisable.
- GET `/api/loans [admin]` : tous les emprunts **en retard** (`dueAt < now` et `returnedAt = null`).
- Limitation du nombre de tentatives de login (rate limiting).
- Remplacer `console.log` par un vrai logger (`pino`).